

Risk-Governed On-Chain DeFi Agents: Deterministic, Regime-Aware Position Sizing Under Real Execution Frictions

Utku Egemen Umut

August 2026

Abstract

Most crypto trading agents built on large language models (LLMs) focus on two questions: what to trade and how to execute it. The question of *how much* to risk, given the current market regime, is usually left to the model or to a fixed rule. This whitepaper describes a small Ethereum DeFi agent, built on the ElizaOS “Spartan” framework, that adds a deterministic risk-governance layer between the language model and execution. The layer sizes positions with volatility targeting, fractional-Kelly shrinkage, and a hard wallet cap, and it refuses Aave borrows that would push the projected health factor below a floor. The sizing arithmetic never runs through the LLM. We verify the execution engine (Uniswap V2 swaps, ERC-20/ETH transfers, Aave V3 supply/borrow) on a mainnet fork with real transaction hashes, and we run a small, no-lookahead sizing backtest on one year of ETH prices to illustrate the layer’s behavior. Our contribution is a proposal and a first implementation, not a result proven at scale; we picked up most of these ideas recently and treat the evaluation accordingly. On a two-month stretch of high volatility, volatility targeting cut realized volatility from about 88% to 60% per year and reduced maximum drawdown from 43% to 35% relative to holding ETH, without claiming any excess return. A follow-up places a language model in the decision loop and executes its choices through the real agent on the fork; across three models from two providers the behaviour is dominated by risk posture rather than model choice, reported as an illustrative result rather than a performance claim.

1 Introduction and the gap

Two recent lines of work frame the problem we care about. **CryptoBench** [1] evaluates LLM agents on expert-curated cryptocurrency analysis and finds a sharp split: agents retrieve facts well but are weak at prediction, with the best complex-prediction score around 18.6%. **LLM multi-agent portfolio management** [2] backtests a team of modality-specialized agents and reports strong returns for the best configuration (about +133.5% at a Sharpe of 1.50 over its test period), but it abstracts away slippage and price impact and does not solve regime switching: its best full-period, best-bull, and best-bear configurations are three different setups, chosen after the fact.

Newer trading-focused benchmarks reinforce the same lesson. **AI-Trader** [4], a live, contamination-controlled benchmark across stocks, A-shares, and crypto, reports that general intelligence does not translate into trading skill and that *risk-control capability* is what determines robustness across markets. **When Agents Trade** [3] and **InvestorBench** [5] draw similar distinctions between agent frameworks, with behavior ranging from aggressive to conservative largely driven by the surrounding framework rather than the model backbone.

The pattern we build on is well known. Direction is close to unforecastable, but volatility is more forecastable: it clusters and is autocorrelated. An agent that is right on direction but three times oversized into a volatility spike still loses. So the useful lever is not better prediction; it is disciplined sizing that leans on the forecastable quantity and respects real execution costs. That sizing-relative-to-regime layer is the part most projects skip, and it is where we try to contribute.

Scope and honesty. This is a whitepaper, not a rigorous study. We are recent learners of these methods, not experts. Every claim below is meant to be proportional to its evidence: the execution engine is verified on a fork with real transaction hashes, and the sizing layer is demonstrated with unit tests and one illustrative backtest. We do not claim profitability, statistical significance, or that the design generalizes.

2 System

The agent is the ElizaOS “Spartan” project adapted to Ethereum. The Ethereum logic lives in a custodial wallet plugin (`src/plugins/multiwallet`). A single file, `utils/ethereum.ts`, holds the viem-based execution engine: address and key handling, Uniswap V2 swaps with automatic allowance and slippage handling, native and ERC-20 transfers, and Aave V3 supply and borrow.

A user imports a wallet in chat, and the agent detects the key, derives the address, and stores it. Chat commands (“swap 0.05 ETH to USDC”, “supply 100 USDC to Aave”) are parsed by the LLM into a structured intent, which is then executed by the deterministic engine. We test against a local mainnet fork (Foundry Anvil), which keeps the hardcoded mainnet contract addresses and real liquidity working while risking no real funds.

On top of this sits the risk-governance layer (`src/plugins/multiwallet/risk`), added as three ElizaOS surfaces backed by one service:

- a **service** that computes the current regime (realized volatility), the volatility-targeted maximum position, and the Aave borrow headroom;
- a **context provider** that injects that budget into the agent on every turn, so the model is aware of its limits and does not propose oversized trades; and
- pre-execution **guards** on the swap and borrow paths that clamp or refuse a request that violates the budget, plus a read-only advisory action that answers “how much can I safely swap or borrow?”.

The important design choice is that the model proposes intent and reads the regime qualitatively, but the multiply-by-size step is deterministic code.

3 Method

The math here is deliberately modest. We reuse standard results and keep the derivations short.

3.1 Uniswap V2 execution and price impact

Uniswap V2 holds reserves (x, y) of two tokens on a constant-product curve $x \cdot y = k$. With the 0.30% fee, swapping an input Δx returns

$$\Delta y = \frac{997 \Delta x y}{1000 x + 997 \Delta x}. \quad (1)$$

The realized (execution) price $p_{\text{exec}} = \Delta y / \Delta x$ is worse than the spot price $p_{\text{spot}} = y/x$, and the gap grows with trade size. We define price impact as

$$\text{impact}(\Delta x) = 1 - \frac{p_{\text{exec}}}{p_{\text{spot}}}, \quad (2)$$

and the engine protects the trade with a slippage tolerance s , submitting a minimum-output bound $y_{\min} = \hat{y}(1 - s)$ where $\hat{y}$ is the quoted output.

3.2 Aave health factor and a borrow floor

For a borrower with collateral values C_i , per-asset liquidation thresholds $\lambda_i \in [0, 1]$, and total debt D , the Aave health factor is

$$\text{HF} = \frac{\sum_i C_i \lambda_i}{D}. \quad (3)$$

A position is liquidatable once $\text{HF} < 1$. We enforce a floor H_{floor} (default 1.5): a prospective borrow of base value b is refused unless the projected health factor $(\sum_i C_i \lambda_i) / (D + b) \geq H_{\text{floor}}$. Inverting this gives the largest safe additional borrow,

$$b^* = \max\left(0, \frac{\sum_i C_i \lambda_i}{H_{\text{floor}}} - D\right), \quad (4)$$

which the advisory action reports directly.

3.3 Volatility targeting and fractional Kelly

Let $r_t = \ln(p_t/p_{t-1})$ be log returns over a trailing window. The annualized realized volatility is

$$\hat{\sigma} = \sqrt{P} \cdot \text{std}(r_t), \quad (5)$$

with P the number of periods per year ($P = 365$ for daily data). Volatility targeting sets the position as a fraction of the wallet so that its risk matches a target σ_{target} :

$$f_{\text{vol}} = \frac{\sigma_{\text{target}}}{\hat{\sigma}}. \quad (6)$$

As realized volatility rises, the fraction shrinks. Independently, the fractional-Kelly fraction for an edge μ and variance σ^2 , shrunk by $\lambda \in (0, 1]$, is

$$f_{\text{Kelly}} = \lambda \frac{\mu}{\sigma^2}. \quad (7)$$

The engine takes the most conservative of the three constraints, including a hard wallet cap c :

$$f = \min(f_{\text{vol}}, f_{\text{Kelly}}, c). \quad (8)$$

When no edge estimate is available, the Kelly term is set aside and sizing rests on volatility and the cap alone.

3.4 Sizing under slippage

Combining sizing with price impact gives a small optimization. If a trade of size q has expected per-unit edge μ and the constant-product curve produces impact that is approximately linear in q near the current reserves (so total impact cost is roughly κq^2), the expected net value is

$$V(q) = \mu q - \kappa q^2, \tag{9}$$

which is concave and maximized at $q^* = \mu/(2\kappa)$. The point is qualitative: the larger the trade relative to pool depth, the more impact erodes the edge, so the size that is optimal net of frictions is smaller than the size a frictionless model would choose. Our implementation does not solve this in closed form; it clamps to the volatility- and cap-based budget, which is the binding constraint in the regimes we tested.

4 Contribution

The contribution is a deterministic, regime-aware risk-governance layer for an on-chain LLM agent, evaluated under real execution frictions rather than in an idealized simulator. Three points distinguish it from the prior work in Section 1:

1. **The LLM is kept out of the sizing arithmetic.** It proposes intent and reads the regime; rules compute the number. This removes a class of errors where a model over- or under-sizes a trade.
2. **Sizing leans on the forecastable quantity.** Following the retrieval-versus-prediction gap in CryptoBench [1] and the risk-control finding in AI-Trader [4], we size on volatility rather than on a predicted direction.
3. **Execution frictions are real, not assumed away.** The multi-agent portfolio work [2] abstracts slippage; here swaps, transfers, and Aave interactions run against real mainnet contracts on a fork, and the borrow guard reads live Aave account state.

We frame this as a proposal plus a first implementation. It is not a proof that the design improves live trading outcomes.

5 Results

All numbers below come from artifacts in the repository; none are invented.

5.1 Execution engine, verified on a mainnet fork

Running the full pipeline against an Anvil mainnet fork produced real transactions. Table 1 lists the confirmed hashes; every receipt has status success. Two batches are shown: a scripted end-to-end smoke test on a fresh fork, and the chat-driven flow where the agent fired the actions itself. A live, read-only call against Ethereum mainnet through the same code returned a Uniswap V2 quote of $1 \text{ WETH} \rightarrow 1876.30 \text{ USDC}$, confirming the read path against real liquidity.

Source	Flow	Transaction hash (truncated)
Smoke test	Transfer (native ETH)	0xc523e4f5...46883
Smoke test	Swap ETH $\rightarrow$ USDC	0x04bbdc3d...77c61
Smoke test	Aave supply USDC	0x277510c2...42b6c
Smoke test	Aave borrow DAI	0x5774d1bd...9a882
Chat-driven	Swap ETH $\rightarrow$ USDC	0x99b65f21...57d04
Chat-driven	Swap ETH $\rightarrow$ USDC	0xee1a4695...90ec9
Chat-driven	Aave supply USDC	0x765bce00...7cc2c

Table 1: On-chain transactions produced on the Anvil mainnet fork (all status success).

5.2 Deterministic sizing and the health-factor floor

The risk layer answers sizing queries directly. For a wallet holding roughly 9999.9 ETH with ETH realized volatility around 39% per year (live), the volatility-targeted budget is capped by the 25% wallet limit at about 2499.97 ETH. For a wallet with \$187.63 of Aave collateral and no debt, Equation (4) gives a safe additional borrow of \$97.57 at the 1.5 floor; a request to borrow \$300 is refused because the projected health factor falls to 0.49. These values are reproduced by unit tests over the pure sizing functions.

5.3 Illustrative sizing backtest

To show how the layer behaves over time we ran a deterministic, no-lookahead backtest on one year of daily ETH prices (13 August 2025 to 12 August 2026), a period in which ETH fell about 58%. The structure borrows from live-agent benchmarks such as AMA [3] and AI-Trader [4]: at each day the policy sees prices only up to that day, chooses an exposure, and that exposure is applied to the next day’s return, with a simple turnover cost. The decision-maker is our deterministic sizing layer, not the LLM. We compare four policies: buy-and-hold, a fixed half-exposure, volatility targeting, and volatility targeting with a full-exposure cap.

Figure 1 and Table 2 report the most volatile 60-day window, where the effect is clearest. Volatility targeting cut annualized realized volatility from 87.6% (buy-and-hold) to about 60%, and reduced maximum drawdown from 42.8% to 35.5%. In the calmest 60-day window (not shown) the volatility-targeted policies stayed near full exposure and tracked buy-and-hold closely, which is the intended behavior: pull risk in when volatility is high, leave it on when volatility is low. Fixed half-exposure achieves low volatility trivially by always investing half; unlike volatility targeting, it does not adapt to the regime.

Policy	Ann. vol	Max drawdown	Sharpe	Excess vs. hold
Buy-and-hold	87.6%	−42.8%	−2.31	0.0%
Fixed fraction	43.8%	−23.8%	−2.31	+16.1%
Volatility target	61.7%	−37.1%	−3.05	+3.8%
Vol. target + cap	59.9%	−35.5%	−2.92	+5.5%

Table 2: Backtest metrics on the volatile window. Sharpe is negative for all policies because ETH fell over the period; the point is volatility and drawdown control, not return.

We read these results narrowly. Volatility targeting does what it is designed to do, which is control realized volatility and drawdown, not generate return. Over the full bear year, plain fixed-fraction sizing had the best return simply because it stayed less exposed; that is not evidence for any policy’s skill, only for lower exposure in a falling market.

Sizing policies on the volatile ETH window (2026-01-20 to 2026-03-21)

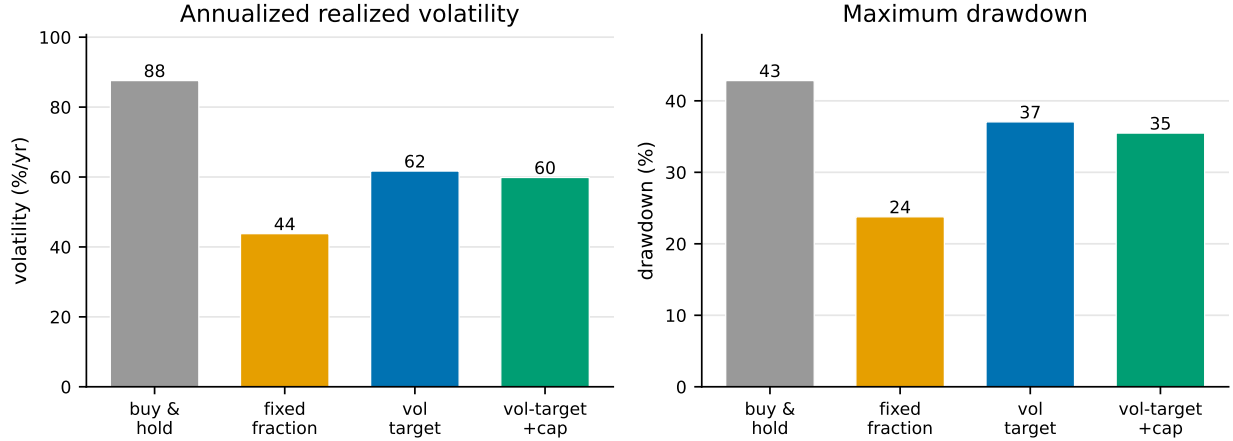


Figure 1: Sizing policies on the most volatile 60-day ETH window. Volatility targeting (with and without a full-exposure cap) lowers realized volatility and drawdown relative to buy-and-hold, while fixed half-exposure does so only by holding a constant half position.

5.4 Related work and where our evaluation falls short

Recent surveys of LLM trading agents are blunt about evaluation quality. A review of 77 studies [7] finds that within the subset that even runs a closed-loop evaluation, only about 10.5% report time-consistent train/test splits and only about 5.3% report an explicit cost model; a companion audit [8] reaches the same conclusion about execution realism. Read against that checklist, our backtest satisfies some items (no lookahead, a turnover cost, a bundled price series for exact reproducibility) and clearly fails others (a single asset, single windows, no multi-seed robustness, and no out-of-sample testing). We report it as an illustration, in the spirit those surveys recommend: preliminary unless supported by stronger evidence. Standardized benchmarks such as InvestorBench [5], STOCKBENCH [6], and CryptoTrade [9] are the natural next targets for a fuller evaluation.

6 LLM-in-the-loop evaluation (follow-up)

As a follow-up we put a language model in the decision loop, in two stages, and we are careful to keep the claims small.

6.1 Deterministic policy comparison

Using the same no-lookahead stepping, we let a model choose a target ETH exposure each day from only past returns and trailing volatility (temperature 0, with decisions cached so runs are reproducible), and compared three models across two providers (`gpt-4o-mini`, `gpt-4o`, `claude-haiku-4.5`). We report three arms: the model’s exposure used directly (*llm-raw*), the model’s exposure clamped to our volatility-targeted budget (*llm+risk*), and the deterministic baselines. Over the same bear year, all three models, prompted to be risk-aware, stayed cautious and cut drawdown sharply relative to holding ETH (about 24%, 28%, and 30% versus 67%), with the model and the provider mattering little. This lines up with AI-Trader’s observation that risk control, not raw model

strength, drives robustness [4]. In this run the deterministic guard did not bind ($llm+risk$ equalled $llm+raw$) because the models never asked for more than the budget the guard enforces; the guard is a backstop against oversizing, and its binding case is the deterministic result in Section 5. Figure 2 shows the drawdown and volatility by arm.

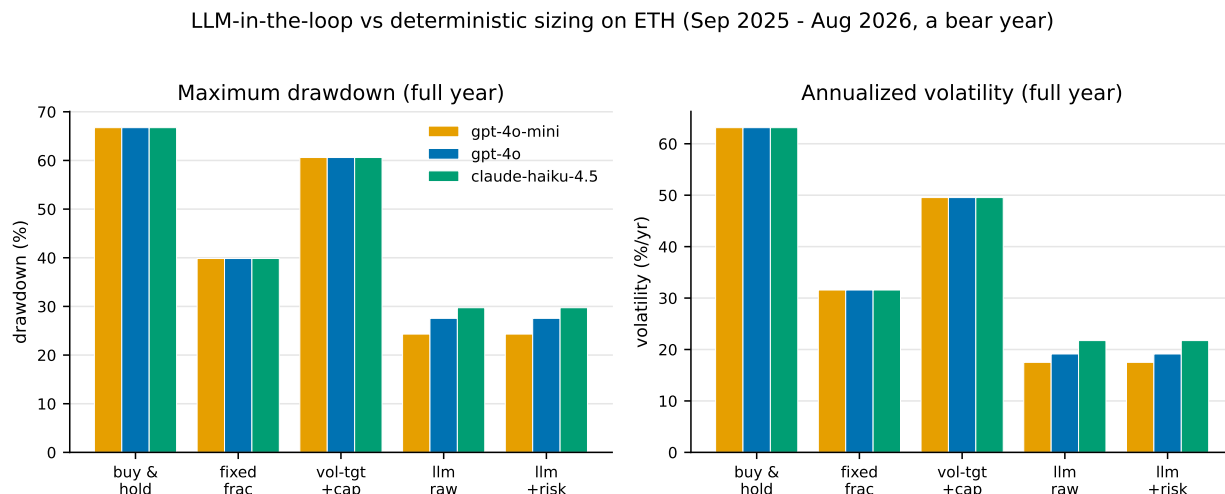


Figure 2: LLM-in-the-loop vs. deterministic sizing on ETH (full year). All three models, prompted to be risk-aware, hold low exposure and so cut drawdown and volatility relative to buy-and-hold; $llm+risk$ equals $llm+raw$ because the guard never binds here.

6.2 Agent-in-the-loop proof of concept

We then replayed the model’s own cached decisions through the real deployed agent: each decision became a chat command that the Spartan agent executed as an actual Uniswap V2 swap on the mainnet fork. Six consecutive decisions produced six confirmed transactions (status success) from the imported wallet. This closes the loop end to end, from a language-model decision to an on-chain action through the deployed agent. It is a capability demonstration at a single forked block, not a historical backtest: the fork does not advance through the six days, and the exposure-to-swap mapping is illustrative. The two stages answer different questions, reproducible measurement and end-to-end execution; details are in `docs/BENCHMARK_LLM.md`.

7 Limitations and reproducibility

The honest caveats are as follows. A fork is not live trading: state is pinned, there are no adversarial counterparties, and MEV is absent. The backtest is one asset over one year with single windows and a single seed. The agent’s action selection is non-deterministic: even when a wallet action is valid, the model sometimes replies conversationally instead of firing it, so a command may need to be repeated. The borrow guard values new debt at roughly one dollar per token, which is accurate for the stablecoin borrows we tested but not in general. And, as stated up front, we are new to this material.

Everything is reproducible. The setup runbook is in `docs/SETUP_FROM_SCRATCH.md`; the pure sizing functions have unit tests (`bun test src/plugins/multiwallet/risk`); the read-only demo,

the fork smoke test, the risk demo, and the backtest are scripts in `scripts/`; and the backtest ships with the exact ETH price series it used so its numbers can be regenerated offline.

Future work is the part we have not earned yet: a multi-seed, multi-asset, out-of-sample evaluation against a standardized benchmark; scaling the LLM-in-the-loop evaluation of Section 6 (more assets, seeds, and a third-party benchmark harness); richer regime inputs (funding, open interest, on-chain flows, macro); and MEV-protected execution for larger orders.

References

- [1] J. Guo et al. *CryptoBench: A Dynamic Benchmark for Expert-Level Evaluation of LLM Agents in Cryptocurrency*. arXiv:2512.00417, 2025.
- [2] Y. Luo et al. *LLM-Powered Multi-Agent System for Automated Crypto Portfolio Management*. arXiv:2501.00826, 2025.
- [3] *When Agents Trade: Live Multi-Market Trading Benchmark for LLM Agents (Agent Market Arena)*. arXiv:2510.11695, 2025.
- [4] T. Fan, Y. Yang, Y. Jiang, et al. *AI-Trader: Benchmarking Autonomous Agents in Real-Time Financial Markets*. arXiv:2512.10971, 2025.
- [5] H. Li, Y. Cao, Y. Yu, et al. *InvestorBench: A Benchmark for Financial Decision-Making Tasks with LLM-based Agent*. ACL 2025; arXiv:2412.18174, 2024.
- [6] *STOCKBENCH: Can LLM Agents Trade Stocks Profitably in Real-World Markets?* arXiv:2510.02209, 2025.
- [7] Y. Xia, P. You, T. Wang, et al. *Agentic Trading: When LLM Agents Meet Financial Markets*. arXiv:2605.19337, 2026.
- [8] J. Yao and Z. Zheng. *Beyond Agent Architecture: Execution Assumptions and Reproducibility in LLM-Based Trading Systems*. arXiv:2606.08285, 2026.
- [9] *CryptoTrade: A Reflective LLM-based Agent to Guide Zero-shot Cryptocurrency Trading*. Findings of EMNLP, 2024.
- [10] H. Adams, N. Zinsmeister, D. Robinson. *Uniswap v2 Core*. Technical whitepaper, 2020.
- [11] Aave. *Aave V3 Technical Paper*. 2022.
- [12] J. L. Kelly Jr. *A New Interpretation of Information Rate*. Bell System Technical Journal, 35(4):917–926, 1956.
- [13] A. Moreira and T. Muir. *Volatility-Managed Portfolios*. Journal of Finance, 72(4):1611–1644, 2017.